3D-Carbon: An Analytical Carbon Modeling Tool for 3D and 2.5D Integrated Circuits

Yujie Zhao[1], Yang (Katie) Zhao[2*], Cheng Wan[1], Yingyan (Celine) Lin[1]
[1]*Georgia Institute of Technology*, [2]*University of Minnesota, Twin Cities*
{eiclab.gatech, chwan, celine.lin}@gatech.edu, yangzhao@umn.edu

Abstract

Environmental sustainability is crucial for Integrated Circuits (ICs) across their lifecycle, particularly in manufacturing and use. Meanwhile, ICs using 3D/2.5D integration technologies have emerged as promising solutions to meet the growing demands for computational power. However, there is a distinct lack of carbon modeling tools for 3D/2.5D ICs. Addressing this, we propose 3D-Carbon, an analytical carbon modeling tool designed to quantify the carbon emissions of 3D/2.5D ICs throughout their life cycle. 3D-Carbon factors in both potential savings and overheads from advanced integration technologies, considering practical deployment constraints like bandwidth. We validate 3D-Carbon's accuracy against established baselines and illustrate its utility through case studies in autonomous vehicles. We believe that 3D-Carbon lays the initial foundation for future innovations in developing environmentally sustainable 3D/2.5D ICs. Our open-source code is available at https://github.com/UMN-ZhaoLab/3D-Carbon.

ACM Reference Format:
Yujie Zhao[1], Yang (Katie) Zhao[2*], Cheng Wan[1], Yingyan (Celine) Lin[1]. 2024. 3D-Carbon: An Analytical Carbon Modeling Tool for 3D and 2.5D Integrated Circuits. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3658482

1 Introduction

For decades, Moore's Law has driven Integrated Circuits (ICs), enhancing their computational power, reducing size and cost, and improving energy efficiency. These advancements have been crucial for developing new technologies like artificial intelligence (AI). However, the carbon emissions from ICs, covering their entire lifecycle from manufacturing to disposal (see Fig. 1), pose significant environmental sustainability challenges. Recent reports indicate that the carbon of Information and Communication Technology represents 2.1%∼3.9% of global greenhouse gas emissions [13].

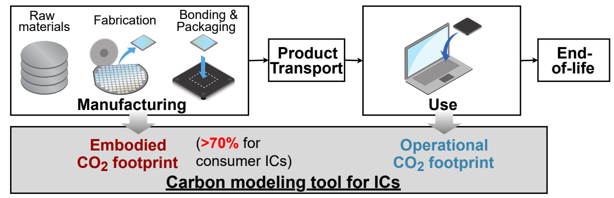

Figure 1. A carbon modeling tool tracks embodied and operational carbon emissions throughout ICs' lifecycle [17].

To ensure environmental sustainability in ICs, addressing their carbon emissions throughout their lifecycle, particularly in *manufacturing* and *use* phases, is crucial [17]. Currently, the *embodied carbon* from manufacturing activities often surpasses the *operational carbon* from energy consumption during the use of today's ICs [17]. This shift is due to extensive operational energy efficiency improvements over years. Embodied carbon can represent over 70% of the total carbon footprint for consumer ICs [11, 17].

In parallel, as the pace of Moore's Law in 2D monolithic ICs has slowed in recent years, significant advancements have been made in developing 3D/2.5D ICs to meet rising computational demands. These advanced ICs offer notable improvements over 2D ICs in power efficiency, performance, and area in various commercial products [12, 19, 23, 30].

However, currently, there's no comprehensive tool to evaluate the carbon footprint of 3D/2.5D ICs, particularly their embodied carbon, which is crucial due to their growing complexity: While additional manufacturing steps increase carbon emissions per wafer, factors like improved yield, area efficiency, use of heterogeneous technologies, and fewer metal layers could reduce the overall carbon footprint.

Current tools for estimating the embodied carbon of 3D/2.5D ICs have limitations. Industry-based databases rely on data about materials [9] or manufacturing processes [11, 17], but their practicality and accuracy are limited by the availability of up-to-date carbon emission data. First-order approaches, such as the one in [10], estimate the embodied footprint per chip based on die size. Another method, ACT+ [11], estimates 2.5D IC carbon footprint from 2D ICs based on cost comparison and simplistically treats 3D stacked dies as 2D. These

* Corresponding author.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3658482

Table 1. 3D/2.5D integration technologies summary.

2.5 or 3D	Integration Technology	F2F or F2B	Stacking	# of max 3D dies/tiers	Representative Manufacturers	Representation Products/ Prototypes
3D	Hybrid Bonding	F2F	D2W/W2W	2	TSMC's SoIC-X [29]	AMD 3D V-cache [30]
		F2B	D2W/W2W	≥2	Intel's Fevores Direct [22]	AMD Ryzen 7-5800X3D [30]
	Micro-bumping	F2F	D2W/W2W	2	TSMC's SoIC-P [29]	Intel Lakefield Core i5-L16G7 [15]
		F2B	D2W/W2W	≥2	Intel's Fevores [22]	HBM [23]
	M3D	F2B	N/A	2		RISC-V Core [19]
2.5D	MCM	N/A	N/A	N/A	AMD's Infinity Fabric [23]	AMD EPYC 7000 series [23]
	InFO	N/A	N/A	N/A	TSMC's InFO-2.5D CoWoS-L/R [29]	AMD Ravi 31 [5]
	EMIB	N/A	N/A	N/A	Intel's EMIB [21]	Stratix 10 [3]
	Silicon Interposer	N/A	D2W	N/A	TSMC's CoWoS-S [29]	Nvidia GPU P100 [26]

techniques provide general insights but lack the detailed breakdowns needed for effective carbon-conscious design.

To comprehensively quantify the overall carbon footprint of 3D/2.5D ICs, we propose 3D-Carbon, an analytical carbon modeling tool, in this paper. Our contributions are as follows:

- We develop 3D-Carbon, an analytical carbon modeling tool for various 3D/2.5D ICs. We believe that 3D-Carbon lays the initial foundation for future innovations in developing environmentally sustainable 3D/2.5D ICs.
- Using 3D-Carbon, we can predict the embodied carbon emissions, including the overhead associated with advanced integration technologies, and estimate the operational carbon emissions during use through surveyed parameters or third-party operational energy estimation plug-ins.
- We evaluate 3D-Carbon and demonstrate its valuable insights and broad applicability through case studies in autonomous vehicles to guide sustainable decision-making in choosing or replacing ICs.

2 Background

2.1 Commercial 3D/2.5D Integration Technologies

We examine three commercial 3D integration technologies and four 2.5D integration technologies (see Tab. 1 and Fig. 2).

2.1.1 3D Integration.

Micro-bumping 3D. This method stacks multiple dies vertically using micron-level solder balls for 3D connections, with a larger pitch than other 3D technologies. **Hybrid Bonding 3D.** This technique stacks two 2D dies using bond pads through the metal layers. **Monolithic 3D (M3D).** M3D employs sequential manufacturing to create fine-pitched MIVs (typically <0.6μm) for inter-tier connections [19]. This paper emphasizes block-level M3D partitioning, where functional blocks like memory and logic macros are separated into different tiers, enabling the use of existing 2D EDA processes [4].

2.1.2 2.5D Integration.

Multi-chip module (MCM). This method places multiple pre-designed dies on an organic package substrate. **Integrated fan-out (InFO).** InFO, evolving from fan-out wafer-level packaging, uses a redistribution layer (RDL) as the substrate, offering smaller line space than MCM and including chip-first and chip-last approaches. **Embedded Multi-die Interconnect Bridge (EMIB).** EMIB integrates multiple dies within a single package using a silicon interconnect bridge. **Silicon interposer.** Utilizing a silicon substrate (passive or active [30]), silicon interposers provide the finest line space but may increase carbon costs.

2.2 IC Carbon and Carbon Optimization Metrics

2.2.1 IC Total Life Cycle Carbon.

Following the industry-endorsed frameworks [11, 17], we estimate an IC's total life cycle carbon footprint as follows, including both operational ($C_{operational}$) and embodied (C_{emb}) emissions:

$$C_{total} = C_{operational} + C_{emb} \tag{1}$$

2.2.2 Indifference Point and Breakeven Time Analysis.

The embodied carbon (C_{emb}) of 3D/2.5D ICs includes both potential savings (e.g., fewer back-end-of-line (BEOL) layers) and overheads (e.g., advanced integration manufacturing processes). Meanwhile, their operational carbon ($C_{operational}$) benefits from shorter interconnect lengths but faces higher power needs for interfaces. Thus, 3D/2.5D ICs don't always offer clear sustainability benefits in both C_{emb} and $C_{operational}$.

To aid in sustainable decision-making for choosing and replacing 3D/2.5D ICs over 2D ICs for fixed workload applications, we use the indifference point metric (T_c) and the breakeven time metric (T_r) as defined in [20], which are based on the embodied carbon costs ($C^{2D}_{emb}/C^{3D/2.5D}_{emb}$), the use-phase carbon intensity (CI_{ues}), and the power consumptions ($P^{2D}_{app}/P^{3D/2.5D}_{app}$, see Eq. (17)):

$$T_c = \frac{C^{3D/2.5D}_{emb} - C^{2D}_{emb}}{CI_{use}(P^{2D}_{app} - P^{3D/2.5D}_{app})}, T_r = \frac{C^{3D/2.5D}_{emb}}{CI_{use}(P^{2D}_{app} - P^{3D/2.5D}_{app})} \tag{2}$$

In scenarios of "choosing", T_c indicates when the saved C_{emb} of 3D/2.5D ICs is offset by increased $C_{operational}$. In scenarios of "replacing", T_r shows the breakeven time when the increased C_{emb} of 3D/2.5D ICs is compensated by reduced $C_{operational}$ with the assumption that the embodied carbon cost has already been invested for the 2D ICs. We compare T_c/T_r to the IC's remaining lifetime (T_{life}) to guide sustainable decision-making in choosing or replacing ICs.

3 3D-Carbon Modeling Tool

3.1 3D-Carbon: High-Level Overview

Fig. 3 shows an overview of 3D-Carbon, concentrating on both the embodied and operational carbon emissions of 3D/2.5D ICs. The relevant parameters have been obtained from industry environmental reports, as listed in Tab. 2.

For estimating *embodied carbon emission*, 3D-Carbon uses a hardware design description consisting of 3D/2.5D configurations and IC area details, a technology information description of the technology node and the maximum number of BEOL layers, and the manufacturing location.

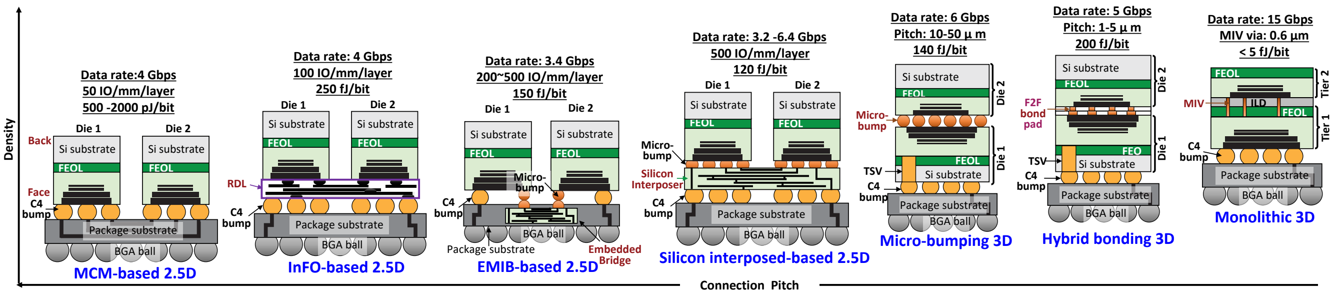

Figure 2. The vertical stack diagram of 3D and 2.5D integration options studied in this paper.

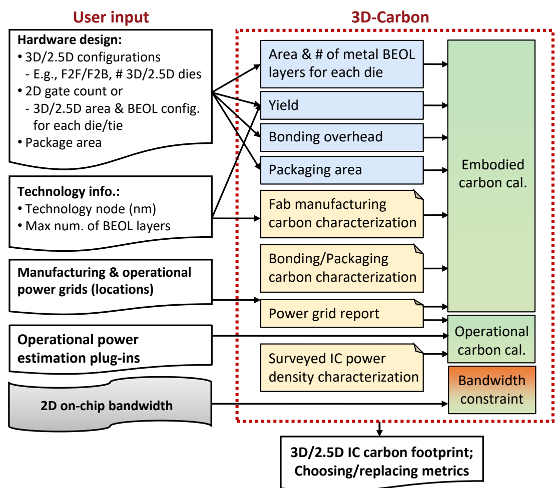

Figure 3. The overview of the proposed 3D-Carbon.

For calculating *operational carbon emission*, 3D-Carbon integrates with operational power consumption plugins like [2, 16, 18] and utilizes the use-location's carbon intensity.

Additionally, 3D/2.5D ICs employ off-die I/O interfaces for data movements, which utilize on-chip wire resources in 2D ICs. Thus, 3D-Carbon introduces an I/O bandwidth constraint to assess the viability of 3D/2.5D ICs compared to their 2D counterparts in terms of data movements.

3.2 3D-Carbon: Embodied Carbon Emission

Unlike 2D ICs, 3D/2.5D ICs integrate N 2D dies with additional bonding emissions. For interposer-based ICs, an extra RDL/silicon interposer is manufactured. We calculate the overall embodied emission ($C_{emb}^{3D/2.5D}$) by summing the emissions from die manufacturing ($C_{die}^{3D/2.5D}$), bonding ($C_{bonding}^{3D/2.5D}$), packaging ($C_{packaging}^{3D/2.5D}$), and the 2.5D interposer ($C_{int}^{2.5D}$):

$$C_{emb}^{3D/2.5D} = C_{die}^{3D/2.5D} + C_{bonding}^{3D/2.5D} + C_{packaging}^{3D/2.5D} + C_{int}^{2.5D} \quad (3)$$

3.2.1 $C_{die}^{3D/2.5D}$ Model. To calculate the embodied carbon from die manufacturing in 3D/2.5D ICs, we begin from the carbon emission per wafer ($C_{wafer_{die_i}}$) for each die (denoted as die_i), the corresponding die-per-wafer count (DPW_{die_i}), and the yield (Y_{die_i}) (as detailed in Sec. 3.2.5):

$$C_{die}^{3D/2.5D} = \sum_{i=1}^{N} \frac{C_{wafer_{die_i}}}{DPW_{die_i}} \cdot \frac{1}{Y_{die_i}} \quad (4)$$

Table 2. 3D/2.5D IC embodied carbon model parameters.

Parameter	Range	Source
Hardware design related parameters		
N_g^{2D}	User input (optional)	Input
N	≥ 2	Input
A_{die_i}	User input (optional)	Input/Eq. (7)
N_{BEOL_i}	User input (optional)	Input/Eq. (10)
Foundary related parameters		
Process	$3 \sim 28\ nm$	Input
$A_{wafer_{die_i}}$	$31{,}415.93 \sim 159{,}043.13\ mm^2$	Input
$Y_{IO}^{micro\_3D/2.5D}$	0~1	[12]
D_{TSV}	0.3 ~25 μm	[19]
GPA_i, MPA_i	0.1~0.5 $kg\ CO_2/cm^2$	[7, 17]
EPA_i	0.4~0.1 kWh/cm^2	[7]
N_{fan}	$1 \sim 5$	[27]
p	$0.6 \sim 0.8$	[27]
β	$450 \sim 850\ M$	[27]
λ	$3 \sim 28\ nm$	[27]
ω	3.6λ	[27]
Bonding related parameters		
$CPA_{RDL}^{InFO_{2.5D}}$	RDL characterization	[7][24]
$EPA_{D2W/W2W}^{micro/hybrid/C4}$	0.9~2.75 kwh/cm^2	[1]
$y_{W2W}^{micro/hybrid}$	0~ 1	[1][23]
Substrate related parameters		
$s_{RDL/EMIB/Si\_int}$	≥ 1	[12]
D_{gap}	0.5~ 2 mm	[12]
Packaging related parameters		
$s_{package}^{3D/2.5D}$	≥ 1	[24][12]
$CPA_{packaging}$	Package characterization	[24]
Carbon intensity		
CI_{emb}, CI_{use}	30~700 $g\ CO_2kWh$	[17]

We calculate the die-per-wafer count (DPW_{die_i}) by dividing wafer area ($A_{wafer_{die_i}}$) by die area (A_{die_i}) [27], similarly applied to interposer-per-wafer count (DPW_{int}):

$$DPW_{die_i/int} = \frac{\pi \cdot \left(A_{wafer_{die_i}}/2\right)^2}{A_{die_i/int}} - \frac{\pi \cdot A_{wafer_{die_i}}}{\sqrt{2 \cdot A_{die_i/int}}} \quad (5)$$

Similar to work [17], 3D-Carbon formulates the wafer carbon footprint ($C_{wafer_{die_i}}$) for die_i as follows:

$$C_{wafer_{die_i}} = (CI_{emb} \cdot EPA_i + GPA_i + MPA_i) \cdot A_{wafer_{die_i}} \quad (6)$$

where CI_{emb} is the carbon intensity of the fab's electrical grid (location), $EPA/GPA/MPA$ is fab energy/gas emissions/raw materials carbon foot print per unit 2D die area.

Area Estimation: Additional areas for Through-Silicon Vias (TSVs) ($A_{TSV_i}^{3D}$) and interface I/O drivers ($A_{IO_i}^{2.5D/Micro\_3D}$) are required in 3D/2.5D ICs for die-to-die transmission. The total die area for die_i includes gate area (A_{gate_i}), TSVs, and I/Os:

$$A^{3D/2.5D}_{die_i} = A_{gate_i} + A^{3D}_{TSV_i} + A^{2.5D/Micro\_3D}_{IO_i} \tag{7}$$

Gate area (A_{gate_i}) is calculated from gate count ($N^{2D}_{g_i}$), feature size (λ), and scaling term (β):

$$A_{gate_i} = N^{2D}_{g_i} \cdot \beta \cdot \lambda^2 \tag{8}$$

$A^{3D}_{TSV_i}$ relates to the size of each TSV (D_{TSV}) and the TSV count (X_{TSV_i}). D_{TSV} corresponds to each technode. Meanwhile, X_{TSV_i} varies depending on the die stacking method (i.e., F2F/F2B). For F2B, the TSV count is calculated using Rent's rule as [27]. F2F TSV count equals the IO number.

Considering the large size of micro-bumps and 2.5D connectors, additional I/O driver area ($A^{micro\_3D/2.5D}_{IO_i}$) is required, calculated using a ratio ($\gamma^{micro\_3D/2.5D}_{IO}$) of the gate area [12]:

$$A^{micro\_3D/2.5D}_{IO_i} = \gamma^{micro\_3D/2.5D}_{IO} \cdot A_{gate_i} \tag{9}$$

BEOL Configuration: Reducing metal layers in the BEOL offers a more environmentally-friendly approach. The number of BEOL layers (N_{BEOL_i}) is estimated as follows [27]:

$$N_{BEOL_i} = \frac{N_{fan} \cdot \omega \cdot N_{g_i} \cdot \bar{L_i}}{\eta \cdot A_{die_i}} \tag{10}$$

3.2.2 $C^{3D/2.5D}_{bonding}$ Model. 3D/2.5D ICs involve wafer-to-wafer (W2W) or die-to-wafer (D2W) integration, stacking multiple dies or wafers [1]. The bounding energy per unit area $EPA^{micro/hybrid/C_4}_{D2W/W2W}$ and yield $Y^{micro/hybrid/C_4}_{bonding_i}$ depends on the choice of D2W or W2W, and on the bonding method (C4-bump, micro-bumping, or hybrid bonding).

$$C^{3D/2.5D}_{bonding} = \sum_{i=1}^{N-1} \frac{CI_{emb} \cdot EPA^{micro/hybrid/C4}_{D2W/W2W} \cdot A_{die_i}}{Y^{micro/hybrid/C4}_{bonding_i}} \tag{11}$$

3.2.3 $C^{3D/2.5D}_{packaging}$ Model. In 3D-Carbon, the packaging carbon footprint is estimated using packaging carbon emissions per area ($CPA_{packaging}$):

$$C^{3D/2.5D}_{packaging} = CPA_{packaging} \cdot A^{3D/2.5D}_{package} \tag{12}$$

The package area $A^{3D/2.5D}_{package}$ is calculated using a linear empirical equation from [12], applying a scaling factor $s^{3D/2.5D}_{package}$, which is based on the largest die area in 3D ICs and the total die area for 2.5D ICs.

3.2.4 $C^{2.5D}_{int}$ Model. The carbon footprint for RDL, EMIB, and silicon interposer substrates is modeled similarly to die carbon footprint. The interposer area calculation differs for each integration method:

$$A_{Si\_int} = s_{Si\_int} \cdot \sum_{i=1}^{N} A_{die_i} \tag{13}$$

$$A_{RDL/EMIB} = s_{RDL/EMIB} \cdot D_{gap} \cdot \sum_{i=1}^{N} l_{adjacent_i} \tag{14}$$

Table 3. Stacking Yields

3D Type	$Y^{micro/hybrid}_{die_i}$	$Y^{micro/hybrid}_{bonding_i}$
D2W	$y_{die_i} \cdot (y^{micro/hybrid}_{D2W})^{N-i}$	$(y^{micro/hybrid}_{D2W})^{N-i}$
W2W	$\prod_{j=1}^{N} y_{die_j} \cdot (y^{micro/hybrid}_{W2W})^{N-1}$	$\prod_{j=1}^{N} y_{die_j} \cdot (y^{micro/hybrid}_{W2W})^{N-1}$

2.5D Type	$Y^{2.5D}_{die_i}$	$Y^{2.5D}_{substrate}$	$Y^{2.5D}_{bonding_i}$
Chip-first	$y_{die_i} \cdot y^{2.5D}_{substrate}$	$y^{2.5D}_{substrate}$	1
Chip-last	$y_{die_i} \cdot \prod_{j=1}^{N} y^{2.5D}_{bonding_j}$	$y^{2.5D}_{substrate} \cdot \prod_{j=1}^{N} y^{2.5D}_{bonding_j}$	$\prod_{j=1}^{N} y^{2.5D}_{bonding_j}$

where s_{Si_int} and $s_{RDL/EMIB}$ are a scaling factors, $l_{adjacent_i}$ is the total length of adjacent sides for all dies, and D_{gap} is the gap between two adjacent dies.

3.2.5 Yield Model. We estimate yields for different process technologies using data from [24] for bonding and packaging yields, and a yield distribution model from [12] for die and substrate yields.

$$y_{die_i} = \left(1 + \frac{A_{die_i}(d,p) \times D_0}{\alpha}\right)^{-\alpha} \tag{15}$$

where D_0 is the defect density and α is a parameter determined by process complexity which both given by [12].

We also consider the impact of each process on the overall yield. Denoting individual process yield as y and overall yield as Y, the yield of a process like W2W bonding ($Y_{bonding_i}$) is affected by its own yield (y_b) and the die yield (y_{die}), as defective dies cannot be separated before bonding. The results for different Y values are listed in Tab. 3.

3.3 3D-Carbon: Operational Carbon Emission

We focus on the fixed-throughput approach, widely adopted in applications like autonomous vehicles (AVs) [28]. Given the varied energy consumption patterns that different power benchmarks can produce, the operational carbon footprint ($C_{operational}$) of ICs is determined by the diverse run-time (T_{app_k}), the use carbon intensity (CI_{use}), and power consumption (P_{app_k}) of each application.

$$C_{opreational} = \sum_k CI_{use} \cdot P_{app_k} \cdot T_{app_k} \tag{16}$$

The power (P_{app_k}) is calculated as:

$$\begin{aligned} P_{app_k} &= \sum_{i=1}^{N} \left(\frac{Th_{app_k}}{Eff_{die_i}} + P_{IO_i}\right) \\ P_{IO_i} &= P_{per\_pitch_i} \cdot N_{pitch_i} \\ N_{pitch_i} &= L_{edge_i} \cdot D_{pitch_i} \cdot N_{BEOL_i} \end{aligned} \tag{17}$$

where Th is the fixed throughput, Eff_{die_i} is the energy efficiency of die_i, and P_{IO_i} is each die's interface I/O driver power. In the absence of specific input for Eff_{die_i}, we utilize surveyed parameters (e.g., as in [19]) to estimate Eff_{die_i}. For 2.5D ICs and Micro-bumping 3D ICs, the I/O power (P_{IO_i}) should be included. We presume P_{IO_i}, using the energy cost per pitch ($P_{per\_pitch_i}$), the length of die_i's edge (L_{edge_i}), the surveyed pitch density (D_{pitch_i}), and the number of BEOL layers (N_{BEOL_i}).

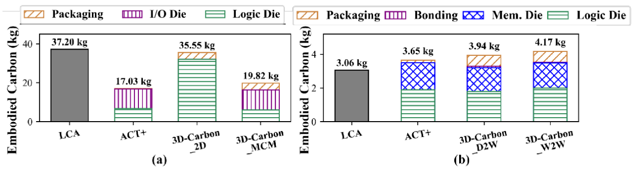

Figure 4. Validation of 3D-Carbon against (a) 2.5D EPYC 7452 [8] and (b) 3D Lakefield [15].

3.4 3D-Carbon: Bandwidth Constraint

A key constraint for 3D/2.5D ICs is ensuring sufficient I/O interface bandwidth for applications. We assume that 3D ICs' I/O bandwidth matches the on-chip bandwidth of their 2D counterparts [6]. The 2.5D ICs' I/O bandwidth is:

$$BW_{die_i} = N_{pitch_i} \cdot BW_{per\_pitch_i} \tag{18}$$

With deep neural networks being the primary workload for AVs [28], we establish the bandwidth constraint that the performance (i.e., throughput) degradation of 2.5D ICs exceeds 20% if the I/O bandwidth is reduced by half than the 2D on-chip bandwidth [6]. Based on this, if the 2.5D ICs' interface causes performance to fall below the throughput requirement, we categorize these instances as “invalid”.

4 Validation of 3D-Carbon

We validate 3D-Carbon by comparing its predicted embodied carbon emissions with those obtained from Life Cycle Assessment (LCA) reports [14] and the latest ACT+ tool [17].

4.1 Validation against One 2.5D IC: EPYC 7452

We validate our 3D-Carbon model against one MCM 2.5D IC, EPYC 7452 [8]. Inputs for both 3D-Carbon and ACT+ are based on the EPYC 7452's specifications: 7nm technology for four CPU dies and 14nm for one I/O die.

As shown in Fig. 4(a), the LCA [14], designed for 2D monolithic ICs, reports higher embodied emissions than 3D-Carbon and ACT+. When we adjust 3D-Carbon for a 2D IC, the discrepancy in embodied emissions between LCA and 3D-Carbon is about 4.4%. Unlike ACT+, our model includes manufacturing complexity: it considers BEOL configurations, adjusting carbon footprint for CPU dies with fewer BEOL layers, and estimates packaging carbon emissions based on actual packaging area, resulting in higher packaging carbon emission (3.47 kg) compared to ACT+'s fixed 0.15 kg carbon.

4.2 Validation against One 3D IC: Lakefield

We validate our model using Intel's Lakefield 3D IC [15], which features heterogeneous integration with a 7nm top logic die and a 14nm base memory die, and show the results in Fig. 4(b). Both 3D-Carbon and ACT+ use the Lakefield's technology specifications.

We reference the GaBi LCA database [14] for LCA validation baseline. Since GaBi doesn't cover the 7 nm process, it assume 14nm for both dies, leading to an underestimation than 3D-Carbon and ACT+ as the 7nm node is more complex and carbon-intensive. Compared to ACT+, our model accounts for manufacturing complexities and differences between D2W and W2W stacking methods. D2W, involving advanced bonding technology, results in lower yield for the bonding process but allows pre-stacking die availability checks, leading to higher individual die yields (Y_{die}). Specifically, the logic die yield in D2W is 89.3%, the memory die is 88.4%, whereas in W2W, both dies have a yield of 79.7%.

Table 4. NVIDIA GPU DRIVE series specifications [25].

	PX 2	XAVIER	ORIN	THOR
Technology node (nm)	16	12	7	5
Num. of transistors (Billion)	15.3	21	17	77
Energy efficiency (TOPS/W)	0.75	1	2.74	12.5
Announced (year)	2016	2017	2019	2022

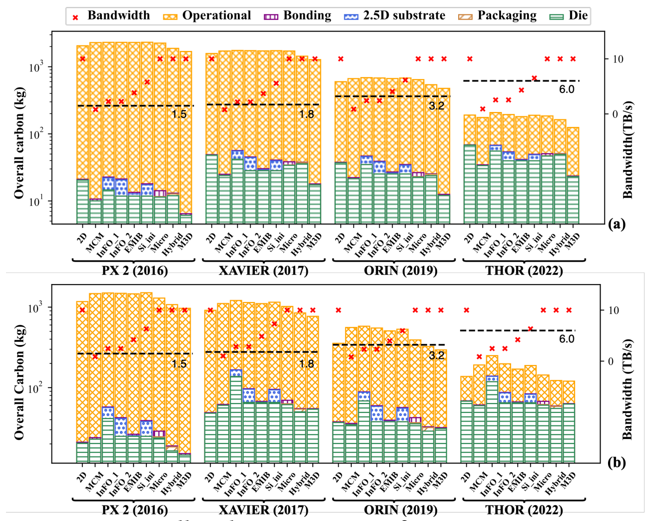

Figure 5. Overall carbon emissions of NVIDIA DRIVE series: 2-die 3D/2.5D ICs with the (a) homogeneous and (b) heterogeneous approaches [6]. Note that InFO_1/InFO_2 represent chip-first/chip-last approaches, respectively.

5 Case study: Sustainable Decision-Making for NVIDIA Autonomous Vehicle GPUs

We conduct sustainable decision-making case studies using 3D-Carbon on the NVIDIA GPU DRIVE series for AVs as detailed in Tab. 4. These compare carbon emissions of original 2D designs with hypothetical 3D/2.5D designs. The hypothetical designs involve two die division approaches: homogeneous (splitting the 2D IC into two similar dies) and heterogeneous (isolating the memory and IOs from the main logic die and implementing them separately in an older 28nm node). For 3D ICs, we consider F2F with D2W stacking.

5.1 Overall Carbon Footprint

Fig. 5(a) and (b) show NVIDIA DRIVE series' carbon emissions for homogeneous and heterogeneous 3D/2.5D approaches, respectively. The black line indicates the required I/O bandwidth, and the red “x” marks achieved bandwidth. InFO and

Table 5. Case studies for choosing/replacing the NVIDIA DRIVE ORIN 2D IC with the 3D/2.5D ICs.

3D/2.5D ICs	EMIB	Si_int	Micro	Hybrid	M3D
Embodied carbon save ratio	23.69%	-9.59%	25.88%	35.64%	65.53%
Overall carbon save ratio	6.5%	-9.86%	7.63%	21.71%	41.03%
Choosing metric T_c (years)	<22	∞	<25	>0	>0
Replacing metric T_r (years)	∞	∞	∞	> 75	> 19

silicon-interposer 2.5D ICs increase embodied carbons due to large substrate areas and low substrate yields. Other 3D/2.5D designs constantly reduce/maintain the embodied carbons, particularly in the homogeneous approach (see Fig. 5(a)), while the heterogeneous approach (see Fig. 5(b)) introduces lesser saving due to smaller memory die areas and limited benefits from the older technology. With the exponential growth of energy efficiency over time, the operational carbon emissions decrease, as detailed in Tab. 4. Operational carbon emissions are higher for 2.5D ICs than 2D/3D ICs, due to the performance degradation (i.e., throughput) from 3D-Carbon's bandwidth constraint (see Sec. 3.4). For THOR, none of the four 2.5D ICs meet the necessary bandwidth, rendering them "invalid".

5.2 Sustainable Decision-Making

We chose the five valid 3D/2.5D ICs with the homogeneous approach for NVIDIA DRIVE ORIN. Tab. 5 presents the embodied carbon savings, overall carbon savings, and metrics for choosing and replacing relative to the original 2D IC. These 3D/2.5D ICs can save up to 65.53% embodied carbon emission and up to 41.03% overall carbon emission. Given the average 10-year lifetime of AV devices, the EMIB 2.5D IC and all three types of 3D ICs can save carbon emissions compared to the 2D IC, as the 10-year lifetime falls within their choosing metric (T_c) ranges. For the decision-making of replacing the 2D IC with 3D/2.5D ICs, we advise against replacing the original 2D IC due to the significant embodied carbon emissions in the new 3D/2.5D ICs, which cannot be compensated by operational carbon savings over their lifetime (see their replacing metric (T_r) ranges).

6 Conclusion

This work introduces 3D-Carbon, an analytical carbon modeling tool designed to understand the carbon emissions of commercial-grade 3D/2.5D ICs at the early design stage. Addressing the need for such tools amidst the growing use of advanced integration technologies, 3D-Carbon aims to pave the way for future developments in environmentally sustainable 3D and 2.5D ICs.

Acknowledgement

This work was supported in part by CoCoSys, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

References

[1] 2023. EVG. https://www.evgroup.com/products/bonding/.

[2] 2023. GPU: Power vs Performance. http://uu.diva-portal.org/smash/get/diva2:767619/FULLTEXT01.pdf

[3] 2024. Intel® Stratix® 10 FPGAs Overview - High Performance Intel® FPGA.

[4] Akgun, , et al. 2019. Network-on-chip design guidelines for monolithic 3-D integration. IEEE Micro (2019).

[5] AMD. 2023. AMD Navi 31.

[6] Arunkumar et al. 2017. MCM-GPU: Multi-chip-module GPUs for continued performance scalability. In ISCA.

[7] Bardon et al. 2020. DTCO including sustainability: Power-performance-area-cost-environmental score (PPACE) analysis for logic technologies. In IEDM. IEEE.

[8] Cristian Constantinescu. 2021. AMD EPYC™ 7002 Series – A Processor with Improved Soft Error Resilience. In DSN-S.

[9] Dell. ,2021. LCA of Dell Servers .

[10] Eeckhout et al. 2022. A First-Order Model to Assess Computer Architecture Sustainability. IEEE Computer Architecture Letters (2022).

[11] Elgamal et al. 2023. Design Space Exploration and Optimization for Carbon-Efficient Extended Reality Systems. arXiv preprint arXiv:2305.01831 (2023).

[12] Feng et al. 2022. Chiplet actuary: a quantitative cost model and multi-chiplet architecture exploration. In DAC.

[13] Freitag et al. 2021. The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations. Patterns (2021).

[14] GaBi. 2022. GaBi database.

[15] Gomes et al. 2020. 8.1 Lakefield and Mobility Compute: A 3D Stacked 10nm and 22FFL Hybrid Processor System in 12×12mm2, 1mm Package-on-Package. In ISSCC.

[16] Guler et al. 2020. McPAT-Monolithic: An area/power/timing architecture modeling framework for 3-D hybrid monolithic multicore systems. VLSI (2020).

[17] Gupta et al. 2022. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In ISCA.

[18] Huang et al. 2010. Interaction of scaling trends in processor architecture and cooling. 198 – 204.

[19] Kim et al. 2021. Micro-bumping, hybrid bonding, or monolithic? A PPA study for heterogeneous 3D IC options. In DAC. IEEE.

[20] Kline et al. 2019. GreenChip: A tool for evaluating holistic sustainability of modern computing systems. Sustainable Computing: Informatics and Systems 22 (2019), 322–332.

[21] Mahajan et al. 2016. Embedded Multi-die Interconnect Bridge (EMIB) – A High Density, High Bandwidth Packaging Interconnect. In ECTC.

[22] Mahajan et al. 2021. Advanced packaging technologies for heterogeneous integration. In IEEE Hot Chip Conference. 22–24.

[23] Naffziger et al. 2021. Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families : Industrial Product. In ISCA.

[24] Nagapurkar et al. 2022. Economic and embodied energy analysis of integrated circuit manufacturing processes. Sustainable Computing: Informatics and Systems (2022).

[25] Nvidia. 2022. Nvidia architecture.

[26] NVIDIA. 2023. NVIDIA Tesla P100.

[27] Stow et al. 2016. Cost and thermal analysis of high-performance 2.5D and 3D integrated circuit design space. In ISVLSI. IEEE.

[28] Sudhakar et al. 2023. Data Centers on Wheels: Emissions From Computing Onboard Autonomous Vehicles. IEEE Micro (2023).

[29] TSMC. 2023. CoWoS.

[30] Wuu et al. 2022. 3D V-Cache: the Implementation of a Hybrid-Bonded 64MB Stacked Cache for a 7nm x86-64 CPU. In ISSCC.

Architectural Whispers: Unveiling Machine Learning Models with Frequency Throttling Side-Channel Fingerprinting

Najmeh Nazari, Chongzhou Fang, Hosein Mohammadi Makrani, Behnam Omidi[2], Mahdi Eslamimehr[3], Setareh Rafatirad, Avesta Sasan, Hossein Sayadi[4], Khaled N. Khasawneh[2], and Houman Homayoun
University of California Davis, [2]George Mason University, [3]UCLA, [4]CSULB
Corresponding author: nnazari@ucdavis.edu

ABSTRACT

Machine Learning (ML) security practices include hiding ML model architectures to protect intellectual property and prevent attacks. We introduce a novel fingerprinting attack using frequency throttling-based Side-Channel Attack (SCA) to detect an ML model's architecture family by converting power side-channel data into timing variations. This method involves using adversary kernels and a time series ML classifier to discern the architecture from execution time patterns during model operation. We achieved up to 96% accuracy in identifying known ML models' architecture families under Ring 0 privileges and we demonstrated its effectiveness across different platforms. Moreover, our code is publicly available [1].

ACM Reference Format:
Najmeh Nazari, Chongzhou Fang, Hosein Mohammadi Makrani, Behnam Omidi[2], Mahdi Eslamimehr[3], Setareh Rafatirad, Avesta Sasan, Hossein Sayadi[4], Khaled N. Khasawneh[2], and Houman Homayoun . 2024. Architectural Whispers: Unveiling Machine Learning Models with Frequency Throttling Side-Channel Fingerprinting. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3657388

1 INTRODUCTION

The significance of ML model architecture knowledge cannot be understated, as it serves as a gateway to numerous security threats, including model inversion attacks and adversarial attacks [17]. In response, the academic community is engaged in the exploration of various channels that could potentially leak information [9, 16].

In this study, we present a breakthrough exploration of ML model architecture fingerprinting attacks through the analysis of power side-channel leakage. Specifically, our approach introduces a novel form of ML model fingerprinting attack that harnesses the passive examination of frequency-throttling side-channel information. This information, recently discovered in [12], is accessible even at the user-space level. We explore the potential of leveraging the frequency throttling SCA and converting the power side channel to timing information to detect the architecture of an ML model performing inference tasks. Our proposed approach overcomes the limitations of telemetry data by employing a black-box attack strategy. This involves the development of a robust supervised classifier using timing information obtained from adversary kernels running concurrently with popular ML models.

Unlike telemetry data, which can potentially be restricted or filtered through software controls, frequency-throttling side-channel leakage cannot be neutralized by imposing access limitations or applying filtering-based mitigation patches. This is due to the fact that any malicious software can monitor its execution time by reading system timers. It is important to note that even low-precision timing information (in order of 100 microseconds) is enough to have high-accuracy fingerprinting results. Furthermore, unlike preceding work [16], our proposed fingerprinting attack does not require access to the victim's system-level trace information such as CPU, GPU, and memory utilization, marking a distinctive contribution in our approach. An additional advantage of our proposed approach, when compared to contention-based timing side-channel (such as execution port) techniques [7], lies in the use of frequency-throttling side-channel leakage which effectively operates across different CPU cores. Moreover, it does not depend on the Simultaneous Multi-Threading (SMT) feature of CPUs to achieve high-accuracy results [18]. This aspect is crucial in cloud environments where SMT features could be disabled on the shared instances [1].

Our proposed ML fingerprinting attack process consists of two key steps: *1) Gathering Timing Information:* We collect the execution time of an adversary kernel while a given ML model is actively running on the same processor, performing inference on a batch of inputs. This allows us to observe the unique timing patterns associated with each ML model architecture. Furthermore, this step involves initiating the frequency throttling side-channel effect and converting it into timing information. *2) Machine Learning Classification:* Following the acquisition of the timing data, we construct an effective supervised classifier customized to discern and accurately identify the architecture of the victim's ML model.

Considering the versatility of ML models, we investigate the transferability of our proposed attack, particularly with unseen model variants—those belonging to the model's family that were not present in our classifier's training set. Our findings showcase the effectiveness of our approach in accurately identifying a known ML architecture family with 98% accuracy. Additionally, when applied to architectures that were never encountered during the training process, our attack still achieved a commendable 89% accuracy in identifying the corresponding architectures' family. Furthermore, to demonstrate the platform portability of the proposed ML fingerprinting attack, we targeted three prominent platforms, including two Intel CPUs, and an AMD CPU paired with an Nvidia GPU.

[1] Access at: https://tinyurl.com/dac24-980

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3657388

2 BACKGROUND AND RELATED WORKS

Frequency throttling, a power management technique adopted on Intel and AMD CPUs, is a result of the dynamic adjustment of the CPU frequency when the execution of a workload causes specific electrical or thermal system parameters to surpass predefined limits. In such situations, the power management architecture reactively takes action to throttle the CPU frequency to a lower value, ensuring the system operates within safe and sustainable conditions [11], while Dynamic Voltage and Frequency Scaling (DVFS) proactively adjusts the voltage and frequency of a CPU in response to real-time workload demands. This study specifically concentrates on the reactive limits that trigger the frequency-throttling side channel as a new source of side-channel information.

Previous studies have utilized various side-channel information to target different attack objectives, including model architecture identification [16], model inputs [6], and parameters extraction [10]. The typical sources of side-channel leakage consist of cache [22], memory access [10], electromagnetic (EM) emission and power [21], timing [6], and GPU statistics [19].

Wei et al. [19] developed an attack called Leaky DNN, which exploits the GPU context-switching penalty to extract the victim's model architecture and it requires access to shared GPU profiling information. In [16], the authors proposed employing shared embedded GPU memory traces on edge devices to identify the model architecture. However, their method can be circumvented with a straightforward memory isolation technique.

Hong et al. [8] utilized the Flush+Reload SCA to extract the DNN architecture by observing specific function calls during inference. On the other hand, Torrellas et al. [22] employed the Flush+Reload and Prime+Probe techniques to monitor special functions, enabling them to infer the model architecture. These techniques are prone to cache partitioning. Cache-based fingerprinting attacks require high-precision timers and access to shared cache. Several defenses have been already implemented against these types of attacks [15]. Batina et al. [4] utilized power and electromagnetic (EM) side-channel to reverse engineer models' architecture. However, collecting EM/power traces requires physical access to the target device.

Table 1 shows a comparison of our work to existing works. Patwari et al. [16] predicts the DNN architecture family based on shared memory usage of CPU and GPU, but this method is vulnerable to isolation defenses and cannot differentiate between specific model architectures. In contrast, EZClone [20] identifies model architectures using GPU kernel features via the PyTorch profiler and, therefore is susceptible to access restriction defenses.

Unlike prior research efforts, our innovative fingerprinting technique can passively and remotely gather timing information exclusively about its own kernel execution time, while a black-box victim ML application operates on a separate core. Consequently, it obviates the need for any GPU profiling data or shared resource usage information. Moreover, we are the first work to fingerprint the transformer-based models (BERT, and ViT).

3 THREAT MODEL

Our proposed approach operates under a closed-world scenario; thus, the attacker possesses knowledge of the potential ML model architectures and the processors running them. Leveraging this awareness, an attacker can create a labeled dataset to train its classifier using a supervised learning technique, in an offline manner.

Table 1: Comparison with related works

Methodes	GPU profiling trace	Shared memory info	Isolation resistance	Access restr. resistance	Target of prediction (Accuracy)
[19]	Yes	Yes	×	×	Model (95.2%)
[8]	No	Yes	×	✓	Model (97.4%)
[22]	No	Yes	×	✓	Model (No report)
[16]	No	Yes	×	✓	Family (99%)
[20]	Yes	No	✓	×	Family/Model (100%)
Our work	**No**	**No**	✓	✓	**Family/Model (96%)**

The primary objective of the adversary in our attack is to determine the victim's ML model architecture (fingerprinting), enabling them to launch a more potent downstream attack. For fingerprinting, the attacker adopts a remote and passive approach, solely measuring its carefully crafted adversary kernel's execution time. If frequency throttling is triggered, the timing information reveals the model architecture.

Our attack is based on four fundamental assumptions:

1) The attacker should have the knowledge of victim's platform. The adversary can use a similar processor to run ML models and collect timing information to mimic the victim's CPU behavior.

2) The victim processor runs one ML model at a time and the inference task is performed on a random batch of inputs (1000 in our case). This is to ensure that the collected timing information reflects the targeted model being executed. For each execution, a new batch of inputs will be loaded. Hence, the probability of using the same image multiple times (for both training and testing) is low. In this way, we make sure that fair sets of inputs are fed to models to prevent input bias on models' behavior.

3) The adversary assumption is that the victim ML model is a derivative of the known families. However, the attacker does not require any prior knowledge of hyper-parameters. The models can even be fine-tuned for specific tasks.

4) The x86 architecture uses four privilege levels (Rings 0 to 3) to implement a form of protection called "privilege level separation". The adversary can operate as a user-space attacker with Ring 3 privilege or as an adversary hypervisor/root-user with Ring 0 privilege, while the victim could be an application running on the same processor (they are not required to run on the same core).

Various isolation techniques can be employed on the system under attack. To initiate frequency throttling, a Ring 3 attacker can run a stressor code alongside the victim application, surpassing the system's default power consumption limits. Alternatively, a Ring 0 attacker can modify the reactive limit of the processor package, prompting the system to trigger throttling activity.

4 PROPOSED METHODOLOGY

4.1 Attack Overview

In our research, we utilize timing information to fingerprint widely used ML model architectures deployed from *Hugging Face* [3]. A typical ML model inference pipeline involves the following steps: 1) Loading the ML model hyperparameters, 2) Loading the inputs to apply the ML model, and 3) The inference task, that entails performing a forward pass of the input through the model to obtain predictions. It is essential to highlight that in our fingerprinting attack, we deliberately mimic a real-world scenario by solely relying on runtime information collected during step 3 of an ML application.

This means that in our experiments the ML model is already loaded into the main memory of the processor and prepared for inference. However, previous works, have relaxed this assumption [16].

Our approach comprises two main phases including the Offline phase and the Online phase. 1) Offline Phase: During this phase, the adversary runs the crafted kernels along with the victim ML model, captures timing information, processes them (normalization and smoothing), and creates a labeled dataset. The collected dataset is then used to train a fingerprinting classifier. 2) Online Phase: In the online phase, the actual attack is executed. The adversary employs the pre-trained fingerprinting classifier from the Offline phase for runtime ML architecture fingerprinting.

4.2 Selection of ML Model Architectures

During the offline phase, the attacker is tasked with creating a dataset that associates the necessary timing information with each known model. For this study, we utilize the *Hugging Face* library and its pipelines. Consequently, we select a pool of machine learning models suitable for image and text classification tasks. Our selection is based on the most commonly used families. To develop a realistic inference pipeline (loading data, loading model, and performing inference), we select the batch of inputs from popular datasets to make sure the dataset used for each model is consistent across models performing the same tasks. For image classification and image segmentation, we used ImageNet-1K and COCO validation sets, respectively. For the text classification task, we used the Yelp review dataset. However, the images used for the *vit-age-classifier* are the images with a person from the COCO dataset.

Table 2 shows the models and families chosen for our research. These selected models serve as the foundation for building the dataset, which will be instrumental in training the ML classifier to effectively identify each ML model's architecture during the online phase of our fingerprinting attack. After selecting the models, the next step in building the dataset is to find a set of features for each model that can be used later for architecture fingerprinting.

4.3 Adversary Kernels

Advanced Vector Extensions (AVX) instructions are power-intensive and can significantly raise a processor's power consumption. Additionally, different types of AVX instructions have varying execution times. This motivates us to use a combination of AVX instructions as an adversary kernel alongside the ML application. With this approach, we can consider the execution time of each kernel as a feature. It is essential to highlight that AVX support is not a mandatory prerequisite, and our methodology remains applicable to platforms that do not possess this parallelism feature. In such instances, a combination of alternative instructions, including *RDRAND*, *RDSEED*, *XSAVE*, *XRSTOR*, and AES-NI instructions (*AESENC*, etc.), can be utilized.

The pathway to identifying each ML model architecture involves establishing a correlation between the execution time of adversary kernels and the architecture of the ML models. However, finding this correlation is not straightforward unless frequency throttling is triggered. For this, our approach relies on the influence of power side channel leakage, which directly affects the timing. This method allows us to ensure that the execution time of the kernel is dependent on the power consumption of other applications running alongside, thereby achieving our objective. This dependency enables us to effectively fingerprint each model based on the unique timing patterns of kernels.

Table 2: Pool of ML models from Hugging Face

#	Model set 1	Family	Task
1	google/vit-base-patch16-224	VIT	Image Classification
2	facebook/deit-tiny-patch16-224		
3	google/vit-hybrid-base-bit-384		
4	nateraw/vit-age-classifier		
5	facebook/convnext-large-224	convnext	
6	microsoft/resnet-50	ResNet	
7	microsoft/resnet-101		
8	microsoft/resnet-152		
9	google/mobilenet_v2_1.4_224	Mobilenet	
10	google/mobilenet_v2_0.75_160		
11	Matthijs/mobilenet_v1_1.0_224		
12	inception_v3.tf_adv_in1k	Inception	
13	inception_v3.gluon_in1k		
14	vgg11.tv_in1k	VGG	
15	vgg13.tv_in1k		
16	vgg19.tv_in1k		
17	nvidia/mit-b0	MiT	Image Segmentation
18	nvidia/mit-b2		
19	distilbert-base-uncased-finetuned -sst-2-english	BERT	Text classification
20	textattack/bert-base-uncased-QNLI		

Table 3: AVX Instruction sets used

	Instruction 1	Instruction 2
Kernel 1	_mm256_mul_ps	_mm256_mul_ps
Kernel 2	_mm256_div_ps	—
Kernel 3	_mm256_mul_ps	_mm256_div_ps
Kernel 4	_mm256_add_ps	_mm256_sub_ps
Kernel 5	_mm256_fmadd_ps	_mm256_fmadd_ps
Kernel 6	_mm256_permute_ps	_mm256_permute_ps

In our approach, we employed a set of six different kernels (implemented in C), each containing a loop with two instructions, as outlined in Table 3. For example, Kernel 4 consists of *add* and *sub* instructions performing the operation on 256-bit vectors containing floats (*ps* stands for packed single-precision). Within each kernel, the loop executes AVX instructions 10,000 times. At the beginning of the loop, the adversary kernel reads the system time, and upon completion of the loop, it calculates the time taken to finish the iteration. This time measurement represents one point of timing data. The kernel repeats this process 20,000 times, resulting in a time series of data consisting of 20,000 data points, which is then saved to a file. Consequently, for each ML model, we possess six traces of timing information obtained from our adversary kernels. These six time-series data collectively form a training sample point for each ML architecture. The collection of these samples creates our dataset. To generate the train set and test set, we use an 80:20 train-test split ratio. We divided our dataset to include 160 samples per model for training and 40 samples per model for the test time.

4.4 Converting Power Side-Channel to Timing information

To convert the power side channel to the timing information, we propose a method that utilizes reactive limit-induced throttling to introduce variations in the execution time of the program. For our specific attack, we induce variations in the execution time of our adversary kernels based on the ML model architecture. In this way, we can effectively convert the power side channel into timing information. This adaptation is crucial to the success of our fingerprinting attack, as it allows us to uniquely identify each ML model architecture based on its power consumption behavior and corresponding timing signatures.

Figures 1 illustrate our proposed approach to convert the frequency throttling side effect into model-dependent runtime information. Let's consider two models, M1 and M2. The model can run on a core separate from our adversary program. Although the adversary cannot directly monitor the execution time of the ML models, it can measure the execution time of its own kernels even with a low-resolution timer.

In the provided examples, the adversary runs three different kernels, each with its unique power profile and execution time. When executing these kernels in parallel with the ML application and if the package power limit is not exceeded, the execution time of each kernel remains the same, regardless of the running model.

However, when the adversary reduces the power limit (Ring 0 privilege) or introduces a power stressor (Ring 3 privilege) to trigger frequency throttling, the execution time of each kernel becomes dependent on the running ML application. This is due to the varied behavior exhibited by each ML model's architecture, which influences their power profiling. Consequently, this leads to time variations in the execution time of the kernels. By observing these time variations in the kernel's execution time, the adversary can determine the specific ML model architecture being executed.

Figure 2 demonstrates how we can exploit the conversion of the power side channel to the timing information for the ML architecture fingerprinting task. To utilize this capability, the adversary kernel must run concurrently with the victim's ML application, and although they do not necessarily have to be on the same core, they should run on the same system. The adversary then triggers frequency throttling by either impacting the reactive limit or running a stress code.

For Ring 0, we followed a specific procedure for each processor. We initially obtained the minimum system power (using the *powerstat* command) when running the given models and kernels. Next, we set the reactive limit to a value lower than the obtained minimum power to ensure that the processor would activate frequency throttling. The new reactive limit is calculated as follows:

$$Reactive\_Limit = \alpha \times Min(Sys\_Power(Model_i, Kernel_j))$$

where α represents a parameter within the range of (0, 1), which we determined experimentally to achieve the most distinct behavior.

For Ring 3, the reactive limit remains configured at its default value. To induce frequency throttling, we needed to execute the stressor program on a minimum of 2/3 of the processor's idle cores. To mitigate any potential interference caused by the stressor program with the operation of the ML application and our adversary kernel, it was essential to restrict its interactions with I/O and minimize its utilization of the main memory and shared cache. To achieve this goal, we employed a specialized matrix multiplication application that was executed in a continuous loop, performing operations on small arrays. The dimensions of these arrays were carefully chosen so that they could fit within the L1 and L2 cache, thereby reducing the need for accessing shared memory resources.

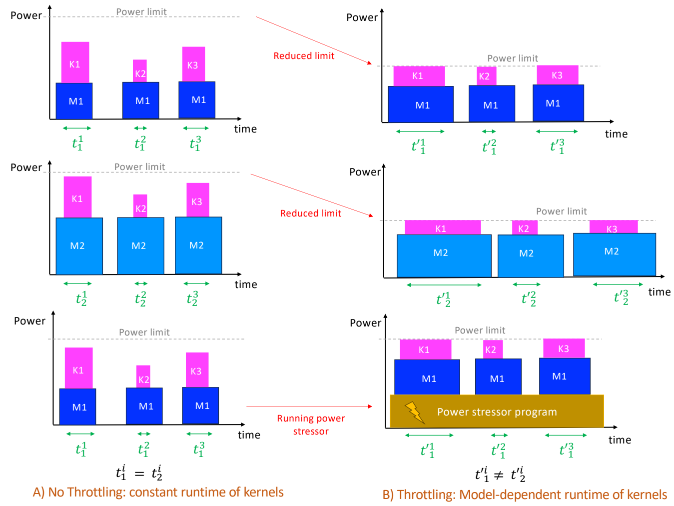

Figure 1: Proposed approach to convert power side-channel to timing information. t_m^k represents the execution time of Kernel k while running alongside Model m.

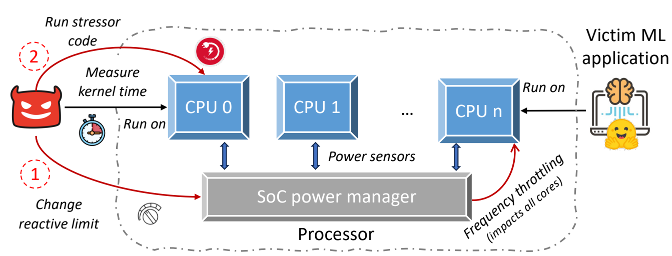

Figure 2: Exploitation of frequency throttling effect. The adversary triggers frequency throttling either by reducing the reactive power limit or running a stressor program.

This approach ensured minimal interference and maximized core utilization, effectively minimizing any disruptions to the primary tasks at hand.

Subsequently, the adversary can run the kernel and measure its execution time. By analyzing these timing patterns obtained during the inference task, we effectively identify distinct timing signatures associated with each model architecture that serves as the basis for identifying the ML models during the online phase.

4.5 Training the Fingerprinting Classifier

The objective of our fingerprinting classifier is to use the collected timing information as input to predict the ML model's architecture and its family when running alongside our adversary kernels. For our classification task, we leverage the *sktime* Python library [2] to construct a machine-learning classifier. This library is a recent addition to the open-source ecosystem, offering compatibility with scikit-learn and providing specialized functionalities for time series tasks. We used the *ROCKET* [5] model, which stands out for its high accuracy and relatively fast performance in handling time series. The dataset for this model must be formatted as follows: 1) Input Samples: The dataset should be in a 3D *numpy* array with

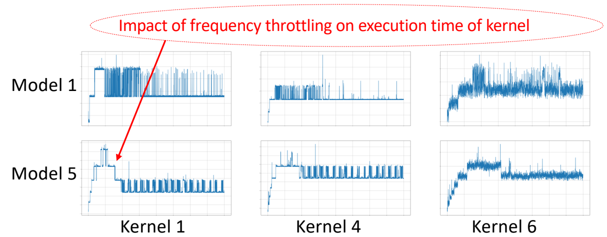

Figure 3: Samples from the dataset for Ring 0 (time series data for 3 kernels running alongside 2 different models).

dimensions (instance, feature, time point). Each instance represents a sample, and each sample contains features observed over different time points. In our study, each kernel's timing trace is considered a feature. Hence we have 6 features over 20,000 time-steps per sample. 2) Labels: The corresponding labels for the input samples should be provided in a one-dimensional *numpy* array. Each element in this array represents the label for the corresponding instance in the input samples. This format ensures that the data is properly structured for input to the *sktime*. The hyperparameter for the *ROCKET* model is the number of random convolutional kernels that we set it to 3,000. Figure 3 illustrates six examples of data, each representing the normalized execution time (Y-axis) of a kernel running concurrently with a specific model for 20,000 consecutive data points (X-axis).

Although the attacker uses the input data on the same device as the victim, it needs to be generalized to handle slight variations. To achieve this, we normalize the timing values between 0 and 1.

5 EXPERIMENTAL RESULTS AND EVALUATION

In our experimental setup, we specifically chose to utilize Intel Xeon CPUs and AMD EPYC processors sourced from x86 machines. To demonstrate the portability of our proposed attack to different platforms, we evaluated the effectiveness of our fingerprinting strategy on two distinct processors with different configurations: 1) Intel Xeon W-2135 CPU, operating at 3.7 GHz frequency, and having a Thermal Design Power (TDP) of 140 Watts. 2) Intel Xeon E5-2650 V2 CPU, running at 2.6 GHz frequency, and having 95 Watts TDP. 3) AMD EPYC 7302 CPU, running at 3.0 GHz with 155W TDP; This platform is equipped with one NVIDIA GeForce RTX 3070 GPU. All systems were equipped with Ubuntu 20.04.6 LTS and Python 3.11. To ensure optimal performance during the experiments, we configured the OS scaling governor to the "performance" mode. In this mode, the clocks are locked at their maximums, unless the power limit is exceeded. This setting is ideal for ML applications on the CPU with low latency.

5.1 Parameter Tuning

To tune the hyperparameter α for Ring 0 attacker, we performed an experiment by sweeping α, and the results are shown in Figure 4.

Based on the optimal accuracy of classifiers, we accordingly set α to 0.8, 0.75, and 0.65 for CPU1, CPU2, and CPU3-GPU, respectively. Once the classification model is trained, the attacker is ready for online deployment. While the ML application is running on the victim's device, the adversary concurrently runs the kernels and measures their execution time. To ensure that frequency throttling is occurring, the adversary may utilize a power stressor or manipulate the reactive limit to trigger throttling.

By collecting the timing information during the execution of the kernels, the trained classifier can be effectively used to pinpoint the architecture of the ML model or identify its family. The unique timing patterns captured during the online phase act as the fingerprint that the classifier uses to accurately distinguish and classify the ML models based on their underlying architecture.

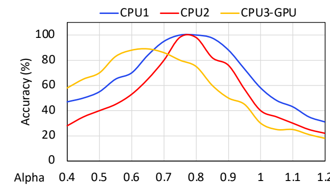

Figure 4: Classifiers' accuracy for various α on Ring 0

Table 4: Pool of ML models as unseen models [3]

#	Model set 2	Family	Task
1	ahishamm/vit-base-isic-patch32	VIT	Image Classification
2	microsoft/resnet-18	ResNet	
3	microsoft/resnet-34		
4	resnet50.tv_in1k		
5	facebook/convnext-large-224-22k-1k	convnext	
6	google/mobilenet_v1_0.75_192	Mobilenet	
7	inception_resnet_v2.tf_ens_adv_in1k	Inception	
8	vgg16.tv_in1k	VGG	
9	nvidia/mit-b1	MiT	Image Segmentation
10	distilbert-base-uncased	BERT	Text classification
11	bert-base-go-emotion	BERT	

5.2 Transferability Analysis

In order to assess the transferability of our attack and investigate its ability to classify unseen models (that are not included in the training dataset), we introduced a new set of 11 models from the selected families (referred to as Model Set 2, as presented in Table 4). The performance of our developed classifier on both Model Set 1 and Model Set 2 is detailed in Table 5. We reported the accuracy of both model classification and family classification.

Model classification accuracy is applicable only to Model Set 1, where the models were used during the training. The results indicate that we achieved an accuracy of over 98% in identifying the architecture's family for the known models in Model Set 1. Furthermore, our classifier successfully classified the family of unknown models in Model Set 2 with an average accuracy of 89.5%. These findings demonstrate the effectiveness of our proposed classifier in classifying both known and unknown model families, underscoring the transferability and portability of our attack.

In Figure 5, we present the confusion matrices corresponding to the family classifier from the CPU3-GPU platform on model set 2. Our analysis reveals that the MiT family demonstrates the most robust classification accuracy. Conversely, the ConvNext family exhibits the least accurate performance. We also note that the VGG architecture exhibits exceptional distinguishability, positioning itself as the second-best performer. Furthermore, our analysis highlights a distinct pattern of misclassifications primarily occurring between

Table 5: Classifiers accuracy results-rounded (%)

	Family classification						Model classification					
Platform	CPU1		CPU2		CPU3-GPU		CPU1		CPU2		CPU3-GPU	
Privilege	R0	R3	R0	R3	R0	R3	R0	R3	R0	R3	R0	R3
Set 1	100	93	99	91	90	83	99	90	95	84	88	77
Set 2	92	86	91	84	89	81	—	—	—	—	—	—

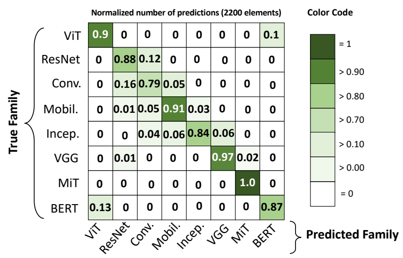

Figure 5: Family classification's confusion matrix: CPU3+GP, model set 2 (200 samples per model), Ring 0

the ViT and BERT models. This pattern is not surprising, given that both ViT and BERT belong to the transformer-based model family, leading to similarities in their classification characteristics.

5.3 Downstream Adversarial Attacks

The objective of adversarial attacks is to deceive ML classifiers by introducing imperceptible perturbations to inputs, undetectable by humans. In this context, we illustrate that our proposed model fingerprinting attack improves the performance of adversarial attacks by transforming the semi-black-box setting into a white-box setting, utilizing knowledge obtained through fingerprinting. In the semi-black-box setting, the attacker constructs a substitute model to generate adversarial examples, with little knowledge about the ML model. In this scenario, the attacker generates adversarial examples using an ensemble of models to enhance their success.

As a case study, we employ pre-trained models from PyTorch and the Adversarial Robustness Toolbox (ART) library [14], for conducting adversarial attacks. Within our experiments, we designate $microsoft/resnet$-101 as the target victim model. Specifically, we utilize the DeepFool attack [13] from the ART library in three distinct scenarios. We use 2,000 images from the ImageNet validation set. In the initial scenario, adversarial examples are generated by using a random ensemble of models. In the second scenario, the attack is orchestrated utilizing models from the same family as the victim. Finally, adversarial examples are crafted specifically for the architecture of the victim model.

Table 6 provides a summary of our findings. The baseline accuracy of the victim model on benign examples is 81.7%. Our results indicate that adversarial examples generated from unrelated model families exhibit lower effectiveness. Leveraging the adversarial samples generated from the ResNet family results in a notable 41% decrease in the accuracy of the victim model. Notably, when considering adversarial examples tailored to ResNet-101, the victim model's accuracy experiences a substantial drop of 68%. Our results show that with the knowledge of the ML architecture, an adversary can generate much more effective adversarial examples.

Table 6: Accuracy drop of models by adversarial examples

Scenario	Models used for adv. examples generation	Accuracy drop (%)
1	VGG19, Inception3, Mobilnet V2	12%
	Convnext_base, VGG13, DenseNet 121	17%
	ResNet 50, VGG16, Convnext_small	23%
2	ResNet 18, ResNet 34, ResNet 152	41%
3	ResNet 101	68%

6 CONCLUSION

This work presents a novel approach to ML models' fingerprinting. By exploiting the frequency throttling side effects, we extract timing patterns through the execution of crafted adversary kernels. Leveraging the time series feature, we develop a classification model to identify the ML model architecture. The experimental results demonstrate the effectiveness of our proposed attack achieving a classification accuracy of 96.3% for known ML model families and 90.6% accuracy for previously unseen models with Ring 0 privilege. By relying on timing information and avoiding direct access to the ML models, we open avenues for stronger remote downstream adversarial attacks on the cloud.

ACKNOWLEDGMENTS

The work in this paper is partially supported by National Science Foundation grants CNS-2155002 and CNS-2155029.

REFERENCES

[1] [n. d.]. *https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/the-ec2-approach-to-preventing-side-channels.html.*
[2] [n. d.]. *https://github.com/sktime/sktime.*
[3] [n. d.]. *https://huggingface.com.*
[4] Lejla Batina et al. 2019. CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium.*
[5] Angus Dempster et al. 2020. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. *Data Mining and Knowledge Discovery* 34, 5 (2020).
[6] Gaofeng Dong et al. 2019. Floating-point multiplication timing attack on deep neural network. In *SmartIoT*. IEEE.
[7] Ben Gras et al. 2020. ABSynthe: Automatic Blackbox Side-channel Synthesis on Commodity Microarchitectures.. In *NDSS.*
[8] Sanghyun Hong et al. 2018. Security analysis of deep neural networks operating in the presence of cache side-channel attacks. *arXiv* (2018).
[9] Xing Hu et al. 2020. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *25th ASPLOS.*
[10] Weizhe Hua et al. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *55th DAC.*
[11] 2009 Intel. [n. d.]. Power Management in Intel Architecture Servers.
[12] Chen Liu et al. 2022. Frequency throttling side-channel attack. In *CCCS.*
[13] Seyed Moosavi et al. 2016. Deepfool: a simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE CVPR.*
[14] Maria-Irina Nicolae et al. 2018. Adversarial Robustness Toolbox v1. 0.0. *arXiv preprint arXiv:1807.01069* (2018).
[15] Daniel Page. 2003. Defending against cache-based side-channel attacks. *Information Security Technical Report* 8, 1 (2003), 30–44.
[16] Kartik Patwari et al. 2022. Dnn model architecture fingerprinting attack on cpu-gpu edge devices. In *7th EuroS&P*. IEEE.
[17] Zirui Peng et al. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *CVPR.*
[18] Daniel Townley and Dmitry Ponomarev. 2019. Smt-cop: Defeating side-channel attacks on execution units in smt processors. In *28th PACT*. IEEE.
[19] Junyi Wei et al. 2020. Leaky dnn: Stealing deep-learning model secret with gpu context-switching side-channel. In *50th DSN*. IEEE.
[20] Jonah Weiss et al. 2023. EZClone: Improving DNN Model Extraction Attack via Shape Distillation from GPU Execution Profiles. *arXiv* (2023).
[21] Yun Xiang et al. 2020. Open dnn box by power side-channel attack. *IEEE Transactions on Circuits and Systems II* 67, 11 (2020).
[22] Mengjia Yan et al. 2020. Cache telepathy: Leveraging shared resource attacks to learn {DNN} architectures. In *29th USENIX Security Symposium.*